Лабораторная работа 1 "Разработка серверного приложения на Node.js"

1. ЦЕЛЬ РАБОТЫ

Приобретение практических навыков создания серверного приложения на платформе Node.js с использованием фреймворка Express.js. Изучение основ маршрутизации, обработки HTTP-запросов различных типов (GET, POST, PUT, DELETE) и формирования ответов сервера **для последующей интеграции с клиентской частью вашего курсового проекта** (начало выполненно в 1 части курса).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы платформы Node.js (установка, модульная система, event loop).
- Принципы работы фреймворка Express.js (создание приложения, маршруты, middleware).
- Методы HTTP (GET, POST, PUT, DELETE, PATCH).
- Работу с параметрами запроса (route params, query string, body).
- Форматы передачи данных (JSON, URL-encoded).

---

3. ЗАДАЧИ РАБОТЫ

- Установить и настроить проект Node.js с Express.js.
- Создать REST API для управления коллекцией данных, **соответствующей предметной области вашего курсового проекта** (например, товары, заявки, пользователи, заказы, задачи и т.п.).
- Реализовать маршруты для CRUD-операций.
- Настроить обработку ошибок и возврат соответствующих HTTP-статусов.
- Протестировать API с помощью браузера, Postman или curl.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Инициализация проекта

1. Создать папку проекта, выполнить `npm init -y`.
2. Установить зависимости: `npm install express`.
3. Установить dev-зависимости: `npm install nodemon --save-dev`.
4. В `package.json` добавить скрипты:
   `"start": "node server.js"`, `"dev": "nodemon server.js"`.

Часть 2. Создание сервера

1. Создать файл `server.js`.
2. Подключить Express:
   `const express = require('express'); const app = express();`
3. Добавить middleware для парсинга JSON: `app.use(express.json())`.
4. Определить порт (3000 или другой) и запустить сервер:
   `app.listen(port, () => console.log('Server running...'))`.

Часть 3. Реализация маршрутов (в соответствии с вашей темой курсового проекта)

Создать набор эндпоинтов:

- `GET /items` – получение списка всех элементов (товаров, задач, пользователей и т.п.).
- `GET /items/:id` – получение одного элемента по ID.
- `POST /items` – добавление нового элемента.
- `PUT /items/:id` – полное обновление элемента.
- `DELETE /items/:id` – удаление элемента.

Данные временно хранить в массиве в памяти сервера. **Название ресурса (`items`) замените на соответствующее вашей предметной области** (например, `products`, `tasks`, `courses`, `orders`).

Часть 4. Обработка ошибок

- При отсутствии элемента возвращать статус 404 и JSON с ошибкой.
- При неверных данных запроса возвращать статус 400.
- Добавить глобальный обработчик ошибок (error-handling middleware).

Часть 5. Тестирование

- Запустить сервер в режиме разработки: `npm run dev`.
- Проверить все эндпоинты через Postman/Insomnia или встроенный REST Client VS Code.
- Зафиксировать результаты.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Титульный лист, цель работы.
- Краткое описание проекта (предметная область вашего курсового проекта, структура маршрутов).
- Листинг файла `server.js` (основные части).
- Скриншоты запросов и ответов для каждого эндпоинта (из Postman или браузера).
- Таблица с перечнем маршрутов, методов, ожидаемыми статусами и примерами запросов/ответов.
- Ответы на контрольные вопросы.
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка `lab21`).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов.**

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. Какие методы HTTP вы реализовали и для каких операций?
2. Как получить параметры из URL (route params) и строку запроса (query string) в Express?
3. Для чего используется middleware `express.json()`?
4. Какой HTTP-статус возвращается при успешном создании ресурса? При успешном удалении?
5. Что такое REST API? Перечислите основные принципы REST.
6. Как организовать глобальную обработку ошибок в Express?
7. Какую структуру ответа сервера вы выбрали для ошибок (например, `{ error: "message" }`)?
8. Как протестировать DELETE-запрос, если ресурс уже был удалён? Какой статус должен быть возвращён?
9. Какие преимущества даёт использование `nodemon` в процессе разработки?
10. В чём разница между методами PUT и PATCH? Какой из них вы реализовали и почему?
11. Как можно добавить поддержку CORS в ваше приложение? (Теоретически)
12. Почему в данном задании данные хранятся в массиве, а не в базе данных? Когда стоит переходить на БД?
13. Какой заголовок `Content-Type` должен быть установлен для отправки JSON-данных?
14. Напишите пример curl-команды для тестирования вашего POST-эндпоинта.
15. Как убедиться, что сервер работает корректно после каждой модификации кода?
16. Какие шаги нужно предпринять, чтобы запустить ваше приложение на другом компьютере?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Это первая серверная лабораторная работа во 2 части курса. Она закладывает основу для следующих работ (подключение базы данных, аутентификация, интеграция с React-фронтендом из 1 части).
- Рекомендуется сразу создавать структуру проекта, которая будет расширяться: `routes/`, `controllers/`, `models/` (даже если модели пока просто массив).
- Название ресурса (`items`) обязательно замените на термин, соответствующий вашей предметной области (например, `products` для интернет-магазина, `tasks` для системы задач, `courses` для платформы курсов).
- Успешное выполнение этой работы позволит в следующих работах подключить базу данных, добавить валидацию и систему аутентификации на основе JWT, а затем интегрировать сервер с вашим фронтендом.